

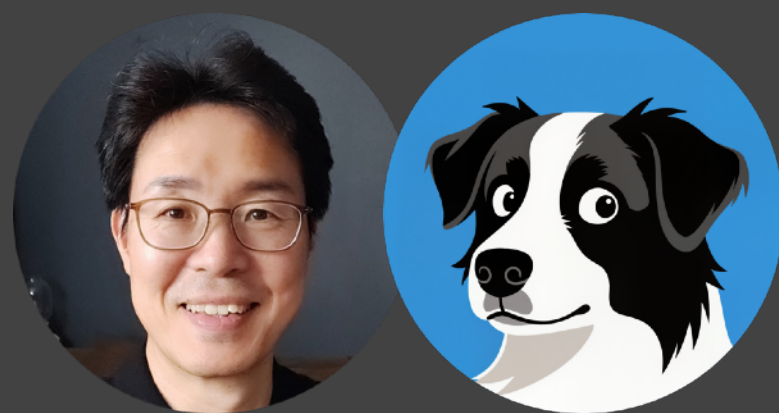
goorm



AI Fit-T 프로젝트 : 라이브코딩 (25.09.05~10.02)

Cursor, React, Git
라이브코딩 20제 마스터

Cursor, React, Git 바이브코딩 20제 마스터



JUN (동준상)

- 넥스트플랫폼 대표 (naebon@naver.com), AX 저술/강연/컨설팅
- KDT 해커톤 (노동부), SW 융합 해커톤 (과기부) 바이브코딩 멘토링
- '데이터 마이닝 개념과 기술 4판', 'AWS SAA 시리즈' 등 20여권 저술
- 구름, KPC 한국생산성본부, SDS 멀티캠퍼스, GSC 글로벌 소프트웨어 캠퍼스 인공지능 및 클라우드 강연 (현대오토에버, KIA, 하나금융 강연)
- 에스24 '이달의 개발자', 삼성SDS SCP 인증자격 출제위원
- 저서: 데이터 마이닝 개념과 기법 4판, 머신러닝 인 자바, 마스터링 자바 머신러닝, AWS 스터디가이드 3, 4판 외 20여권 저술 및 번역

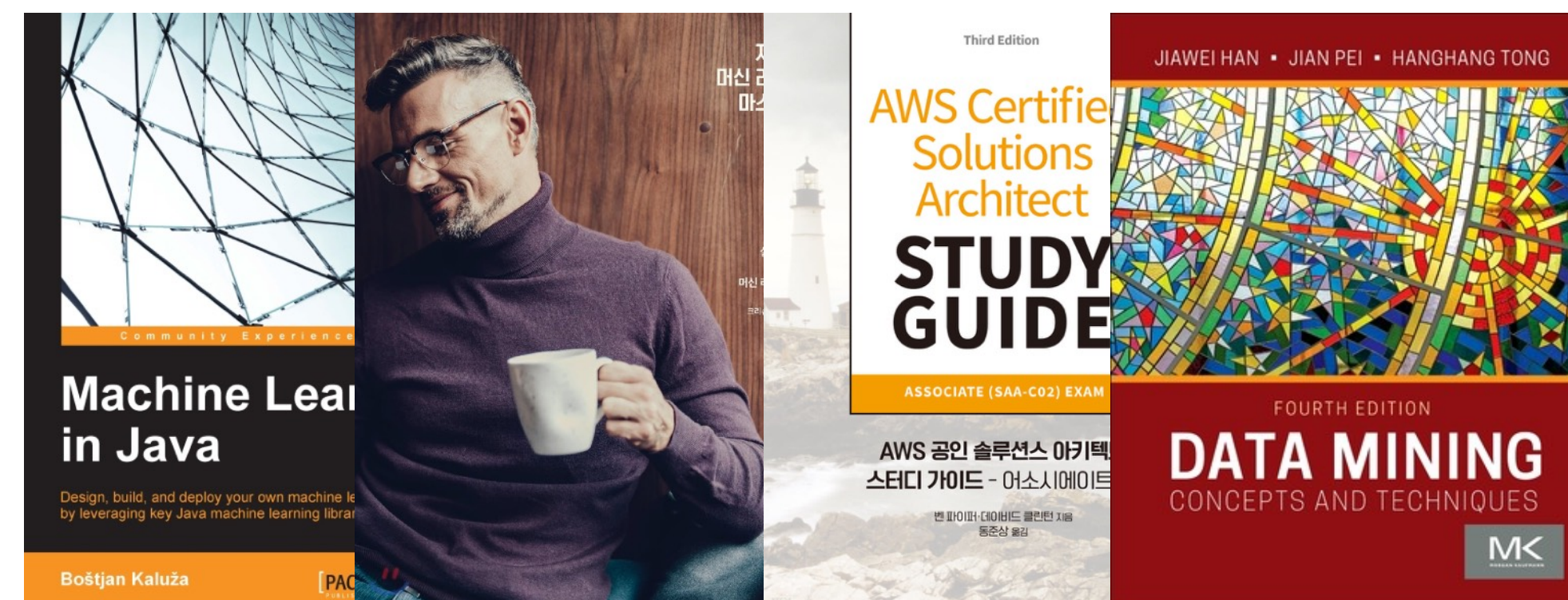
+

강연기관

SAMSUNG SDS



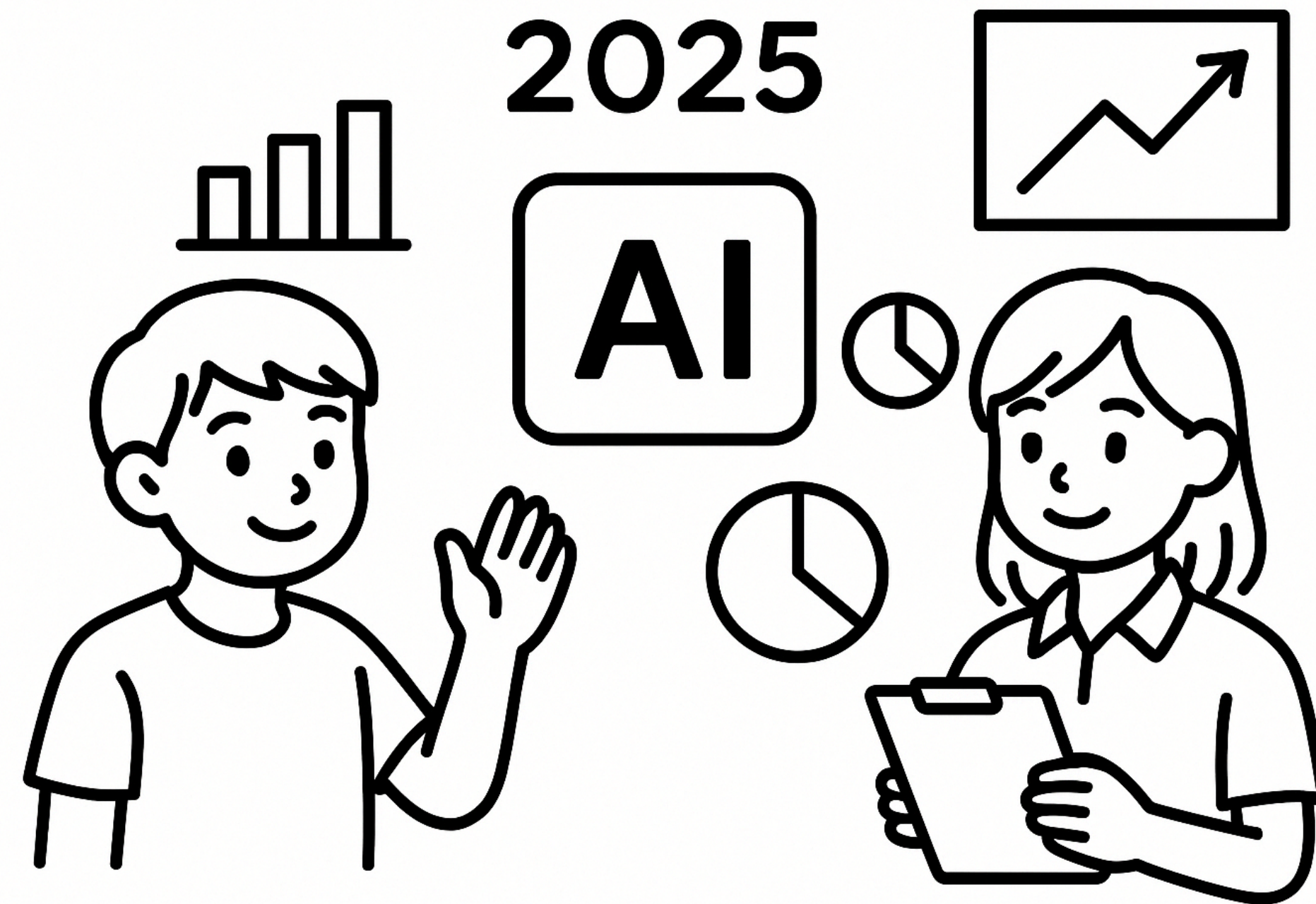
저서/번역서



<https://nextplatform.net/동준상-넥스트플랫폼-프로필/>

목차

1. 바이트코딩 시작하기
2. 입문자 바이트코딩 도구
3. 모범적인 바이트코딩 방식
4. 바이트코딩 입문 프로젝트
5. 바이트코딩 응용 프로젝트
6. 바이트코딩 점프 스타트
7. 바이트코딩 실전 프로젝트
8. 바이트코딩의 한계영역



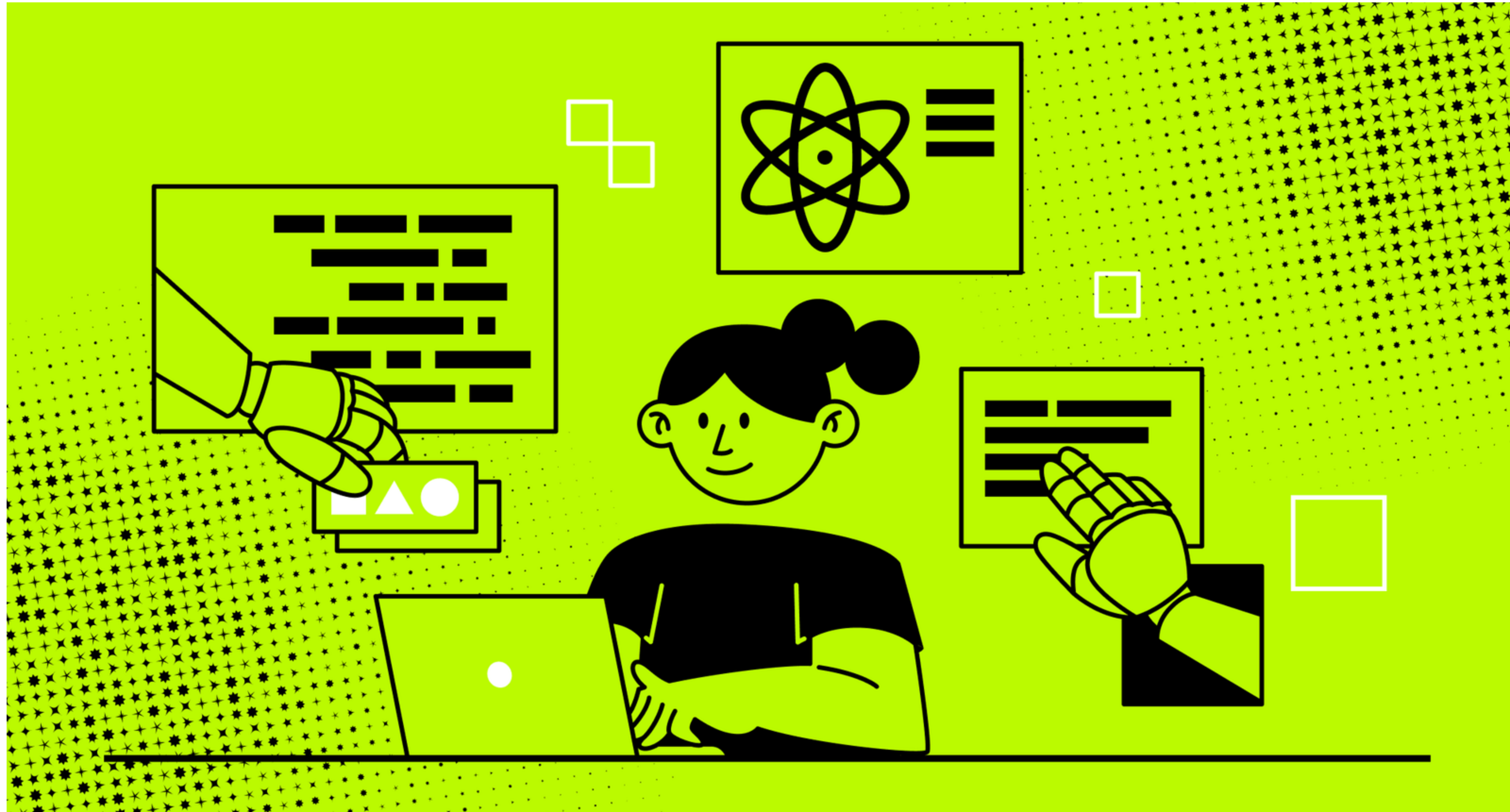
VibeCoding Master

바이브코딩 시작하기

(Getting Started VibeCoding)



바이트코딩 세계로 오신 것을 환영합니다.

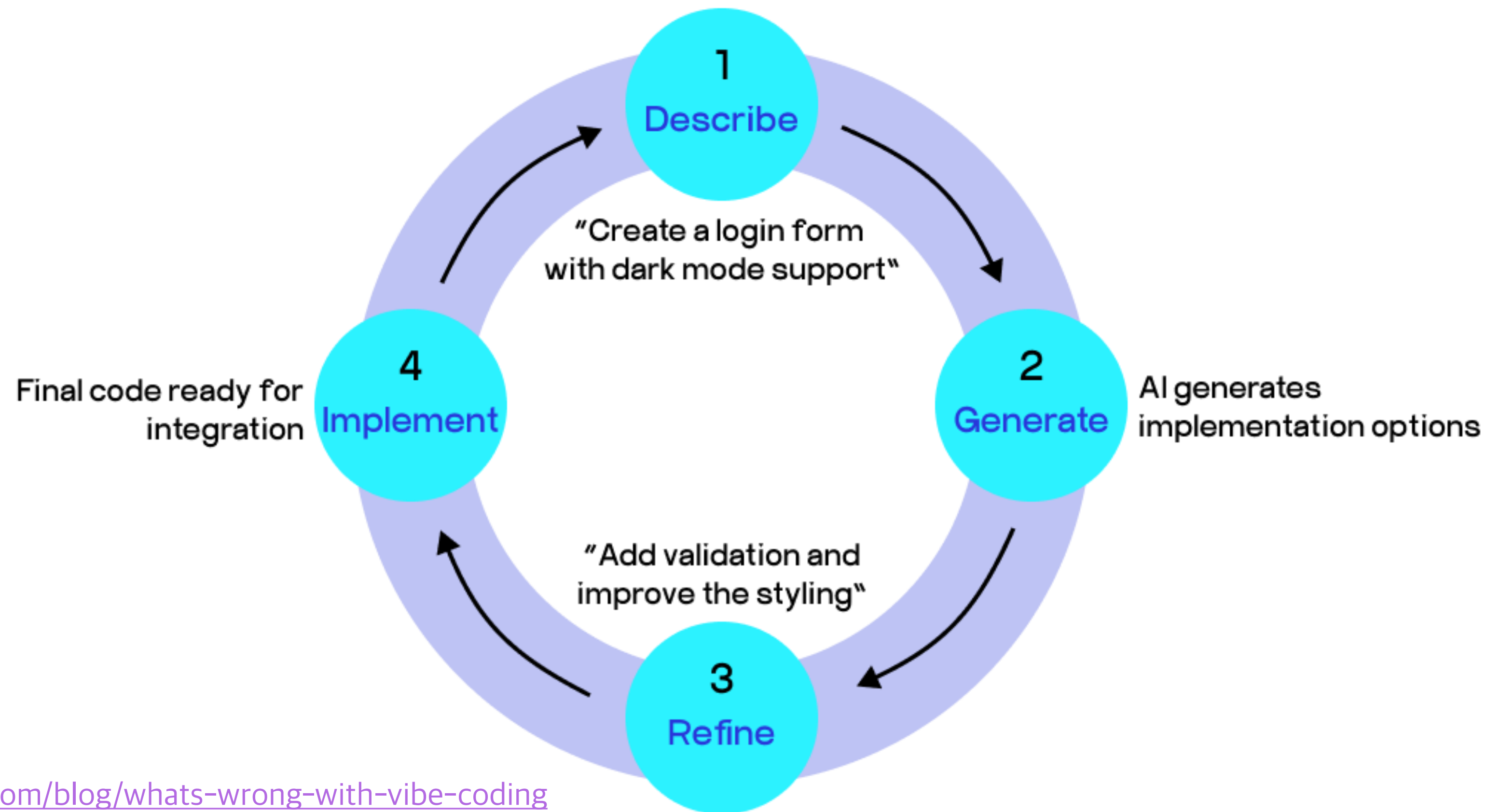


시작하기: 바이트코딩이란?

- AI와 함께하는 실시간 프로그래밍 방식
- 전통적 코딩 지식 → AI 활용 역량으로 재분배
- 문맥 관리, 코드 검증, 반복 개선 중심
- 목표: 기획→개발→배포의 전 과정을 AI와 협업

바이트코딩 4대 프로세스

The vibe coding workflow



바이트코딩과 기존 코딩 방식 비교

Aspect	Vibe coding	Traditional coding
Required skills	Communication ability, clear explanation of needs	Programming languages, algorithms, debugging
Development speed	Rapid prototyping, results in hours or days	Slower development cycle with thorough planning
Code understanding	Limited knowledge of how the code works	Complete understanding of implementation details
Maintenance	Challenging to modify without AI assistance	Easier to adjust, debug, and optimize
Learning investment	Minimal technical learning needed	Months/years mastering programming fundamentals
Project scale	Better for smaller projects and prototypes	More suitable for complex, enterprise applications

바이트코딩 기획1: 목표 설정

- 서비스 목적과 핵심 기능 정의
- 기술 스택, 인증, 수익화 전략 구상
- 초기 다이어그램/워크플로우 작성
- AI 제안 → 의도 수정/구체화 가능

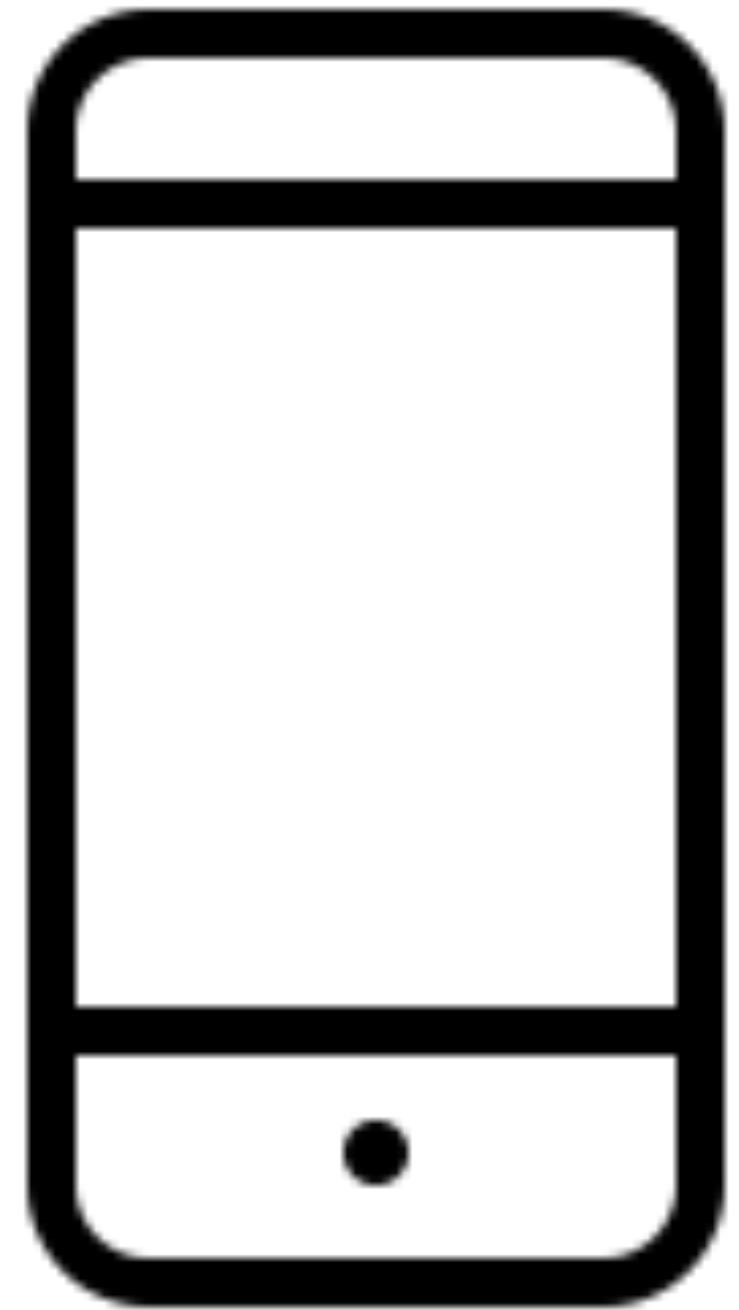
기획 예시0: GPT에 요청

영어 문화권 사용자를 대상으로 iOS용 한국어 학습 앱을 만들 계획이야.
한국어 학습 콘텐츠는 30초에서 90초 길이의 유튜브 영상 클립을 URL로 연결할 수 있게 해줘.
위 작업을 위한 모범적인 바이트코딩 업무 흐름을 제안해주겠어?



기획 예시1: 목표, 전략

- 앱 콘셉트: 영어권 사용자 대상 한국어 학습
(30-90초 YouTube 클립 활용)
- 핵심 가치: 짧고 몰입도 높은 학습 → 발음·표현
- MVP 범위: 영상 재생, 이중 자막, 표현 카드
- 바이브코딩 원칙: 아이디어 → 프롬프트 → 코드 → 실행 → 검증 (하루 단위 루프)



기획 예시2: 시스템, 데이터 스키마

- iOS: SwiftUI + WKWebView(iFrame)
- 백엔드: Supabase (콘텐츠/진행 저장)
- 분석: Firebase Analytics
- clips: 유튜브 URL, 제목, 주제, 난이도
- cards: 표현/의미/로마자/태그
- quizzes: MCQ/번역/순서 배열



기획 예시3: 콘텐츠 & UX 설계

- 콘텐츠 파이프라인: 영상 큐레이션 (저작권)
- 자막 확보 → 카드 & 퀴즈 자동 생성 (LLM)
- UX 요소: KO/EN 자막 + 하이라이트 싱크
- 카드 스와이프 학습 + 퀴즈 3문항
- 프롬프트 예시: “자막에서 표현 5개 추출
→ {ko, romanization, en} JSON 카드 생성”



바이트코딩 기획2: 도구, 모델 선택

- 프로젝트 성격에 맞는 IDE 선택 (Cursor 등)
- 모델별 장단점 비교 (속도/품질/비용)
- 다양한 AI 모델 혼합 사용 (Claude, Gemini 등)
- 실제 사용 사례: Cursor 선택 이유

바이트코딩 기획3: 프롬프트 준비

- Code Guide 등 메타 도구 활용
- 요구사항을 문서화하여 환각 최소화
- 시스템 지침 수정: 언어·프레임워크 지정
- 프롬프트 사전 처리로 AI 품질 향상

바이트코딩 개발1: 반복적 사이클

- (1) 목표 설정 → (2) 코드 생성 요청
- (3) 코드 검토 → (4) 수락/거부
- (5) 테스트 실행 → (6) 문제 식별
- (7) 프롬프트 개선/수동 디버깅
- 이 과정을 반복하며 완성도 상승

바이트코딩 개발2: 프롬프트 전략 ①

- 문맥 초기화: 새 창/탭으로 집중 관리
- 함수·API 이름 등 구체적 참조 활용
- 구조화된 지시: 번호 매기기/조건 제시
- JSON 예시, 오류 메시지 제공 → 품질 향상

바이브코딩 개발3: 프롬프트 전략 ②

- 외부 문서 URL 첨부 → 신뢰도 향상
- 프롬프트 범위 제한(“Stripe는 나중에”)
- 고수준→세부 정보로 점진적 상세화
- 복잡도 단계별로 세분화된 요청

바이브코딩 개발4: 코드 검토

- AI 코드 전체 구조(gestalt) 빠른 파악
- 시각적 diff로 변경 사항 즉시 확인
- 주요 함수 호출·주석·API 사용 확인
- 실행 전 코드 의도와 동작 예측 가능

바이트코딩 개발5: 디버깅 전략

- 하이브리드: AI + 수동 병행
- 브라우저 콘솔/네트워크 탭 활용
- 오류 메시지 AI에 직접 전달 후 수정 요청
- 모델 전환(Claude ↔ GPT ↔ Gemini) 활용

바이트코딩 개발6: 수동 개입

- 간단 수정 → 직접 편집이 더 효율적
- 사소한 UI 요소 정제·삭제 시 직접 변경
- 복잡한 버그 진단 → 전통적 디버깅 의존
- 통제력 유지 차원에서 수동 검토 선호

바이트코딩 배포1: 품질 관리

- 코드 복잡도·유지보수성 최종 점검
- 설계 표준 및 의도 적합성 확인
- 실제 운영 환경 통합 여부 확인
- PR 검토 후 배포 권장

바이브코딩 배포2: 문서화

- 프로젝트 계획·문서 최신화 필요
- 버전/라이브러리 정보 확인 & 수정
- 실제 코드와 문서의 불일치 보정
- AI 생성 문서 → 최종 수동 검증 필수

바이브코딩 체크리스트1: 전문성

- 전통적 코딩 지식은 여전히 중요
- AI 모델 한계·비용·성능 이해 필요
- 제품 관리 역량: 목표→기능 변환
- 새로운 역량: “주변 역량(Ambient Competence)”

바이브코딩 체크리스트2: 신뢰 관리

- 맹목적 수용 X → 지속적 검토 필요
- 존재하지 않는 API 제안 거부
- 개인 전문성에 따라 신뢰도 조절
- 신뢰 = 반복 검증을 통한 관계 구축

바이트코딩 체크리스트3: 문맥 관리

- 새 세션/탭으로 문맥 초기화
- 초반 프롬프트 → 프로젝트 방향 좌우
- 필요 시 과감히 재설정
- 문맥 모멘텀(Context Momentum) 관리

바이브코딩 체크리스트4: 프롬프트

- 고수준 + 상세 지시 혼합 활용
- 구조화된 지침/예시/외부 문서 포함
- 환각 줄이기 위해 범위 제한
- 점진적 구체화로 명확성 확보

바이브코딩 체크리스트5: 디버깅

- AI = 보조 도구, 전통 디버깅 병행
- 오류 메시지 분석·터미널 활용
- 복잡 버그 → 직접 가설·수정 지시
- AI에 “구체적 수정” 요청하기

바이브코딩 체크리스트6: 수동 개입

- 효율성: 사소한 변경은 직접
- 정제: AI 코드 보완은 수동 편집
- 복잡 버그: 직접 진단 후 AI에 실행 지시
- 개발자 통제력 유지의 핵심

바이브코딩 체크리스트7: 사회적 영향

- AI 사용 = 부정적 인식 가능성 존재
- 투명하게 AI 활용 과정 공유 필요
- 기술적 전문성·디버깅 능력 강조
- “AI를 잘 안내하는 역량” 시연

정리: 바이트코딩 실무 적용 가이드

- 기획→개발→배포: AI와의 협업 최적화
- 인간 전문성 = AI 활용 성공의 필수 요소
- 신뢰/문맥/프롬프트/수동 개입 → 4대 원칙
- 기획자의 지식, 의도, 경험을 담아 / AI의 폭 넓은 통섭 능력을 활용하여 / 사용자 친화적인 소프트웨어 기능을 구현

VibeCoding Master

입문자 추천 바이브코딩 도구

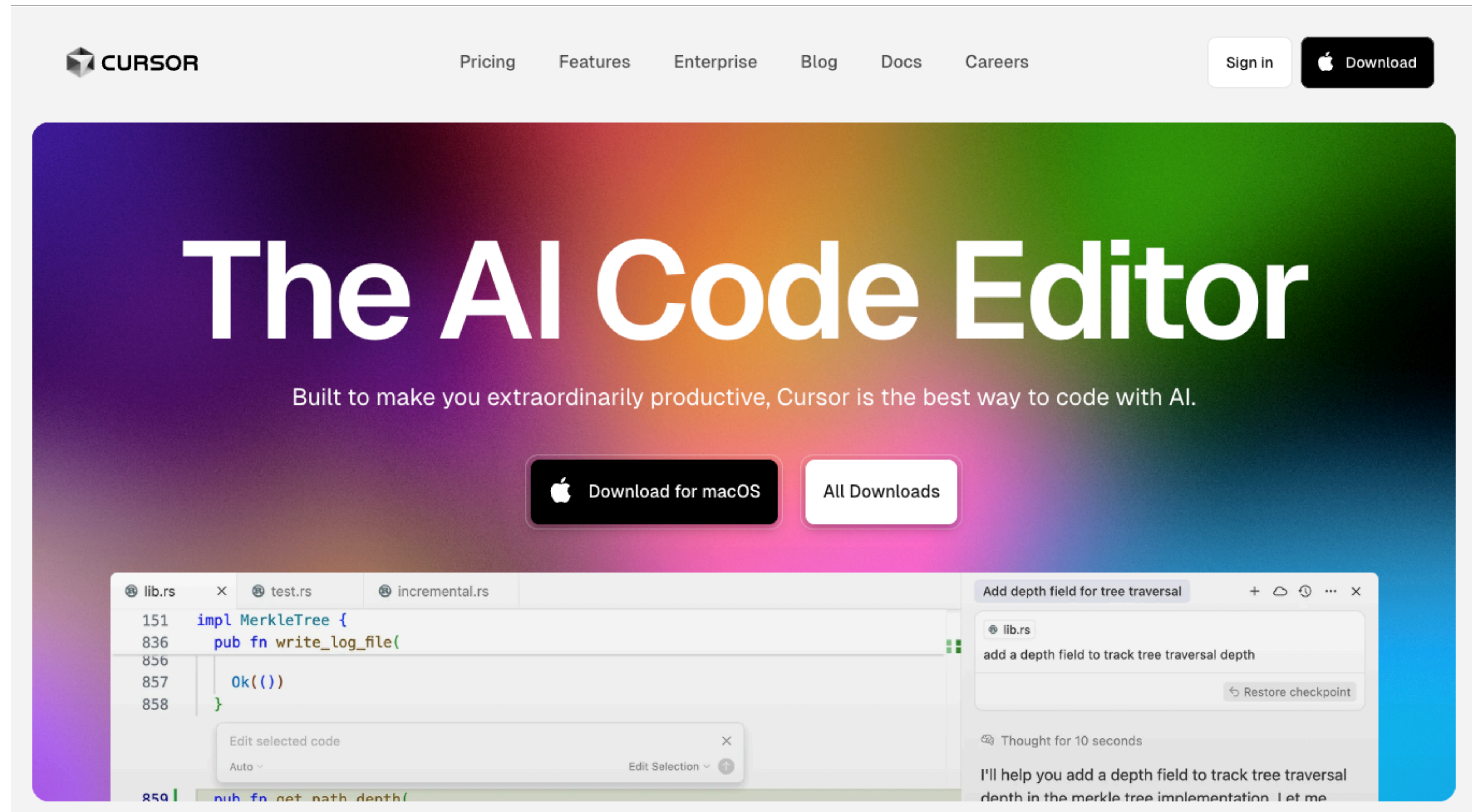
(Best Tools for VibeCoding Beginner)



1. Cursor

- VS Code 기반 IDE, AI 코드 작성·리팩토링·검색 최적화
- GPT/Claude/Gemini 모델 자유 연결
- 프로젝트 단위 System Prompt & Rule 관리
→ 바이트코딩 핵심 툴
- 웹/앱, 자동화, SaaS 템플릿까지 확장성 최고
- MCP로 외부 API·DB 연동

1. Cursor



1. Cursor



영어 ▾

Q 찾다...

₩ K

🌟 AI에게 물어보세요

로그인

다운로드 >

⚙️

선택 서류 비치

CLI

가이드

도구

🌐 웹사이트

📄 법정

🗣️ 지원하다

시작하기

환영

설치

빠른 시작

개념

핵심

꼬리표

대리인 >

백그라운드 에이전트

백그라운드 에이전트 API >

웹 및 모바일

인라인 편집 >

구독

시작하기

빠른 시작

5분 안에 커서를 시작하세요

이 빠른 시작에서는 커서의 핵심 기능을 사용하는 프로젝트를 안내합니다. 이 과정을 마치면 탭, 인라인 편집, 에이전트에 익숙해질 것입니다.

커서에서 프로젝트 열기

기존 프로젝트를 사용하거나 예제를 복제하세요.

복제 예제 프로젝트

기존 프로젝트 사용

1. git이 설치되어 있는지 확인하세요

2. 예제 프로젝트를 복제합니다.

```
git clone git@github.com:voxelize/voxelize.git && \
cd voxelize && \
cursor .
```

📄

🌟

이 페이지에서

커서에서 프로젝트 열기

탭으로 자동 완성

선택 항목 인라인 편집

에이전트와 채팅

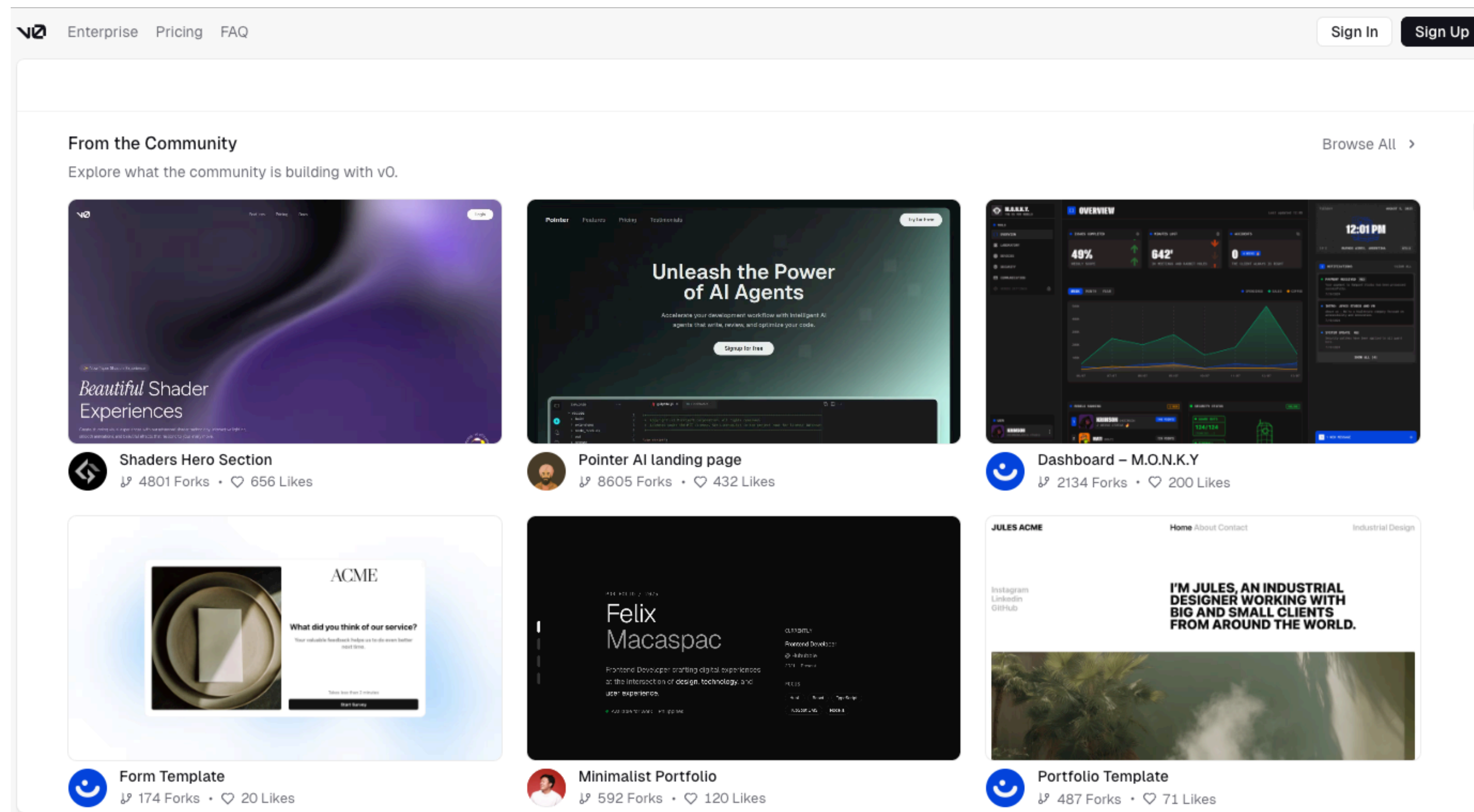
보너스

다음 단계

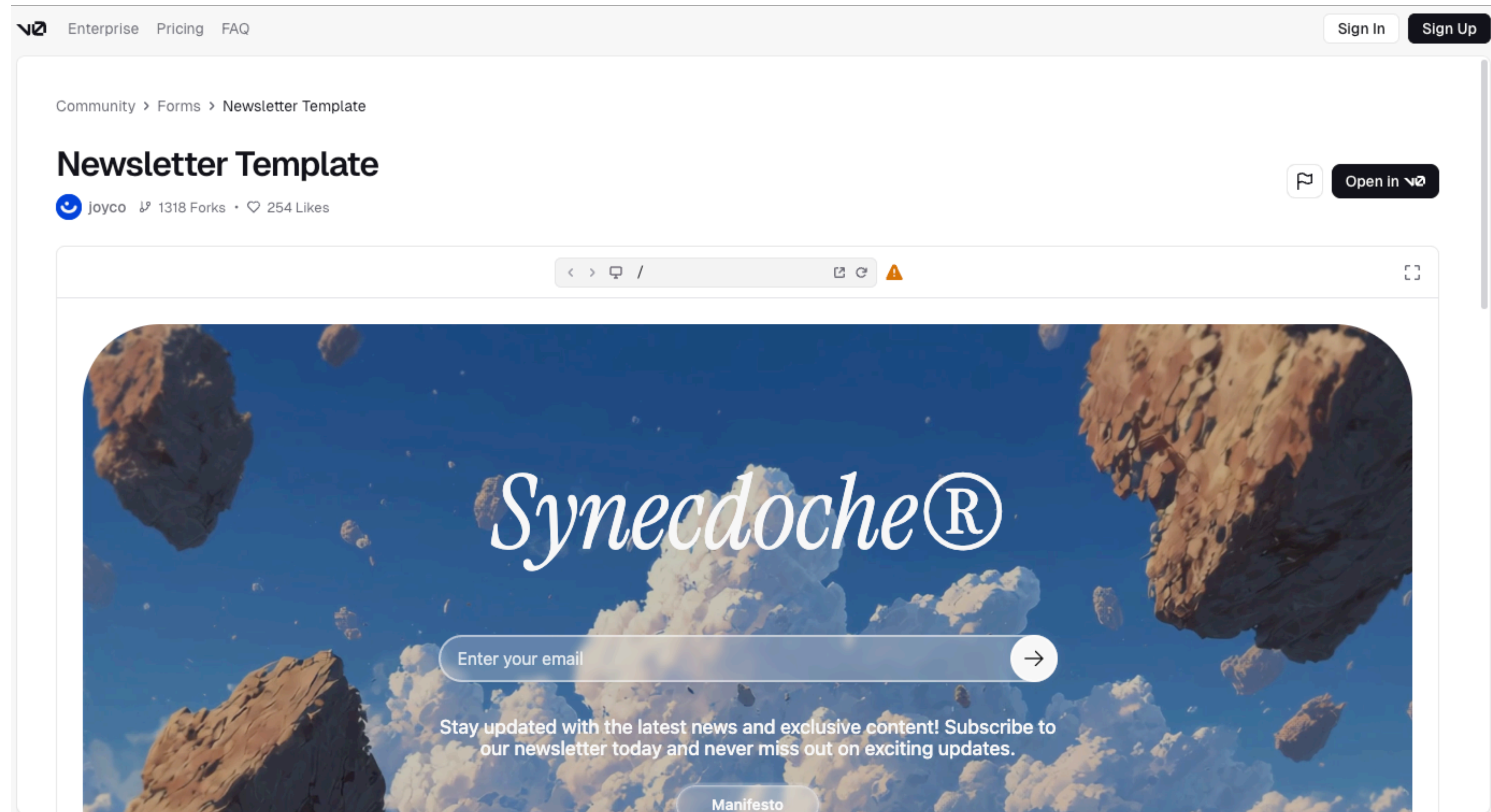
2. v0 (by Vercel)

- 프롬프트만으로 UI 컴포넌트를 코드로 생성하는 서비스
- React/Tailwind 기반 UI 자동 생성
- 디자인→코드→배포까지 빠른 루프 지원
- 입문자도 “프롬프트 한 줄”로 실행 가능한 웹페이지를 곧바로 배포

2. v0 (by Vercel)



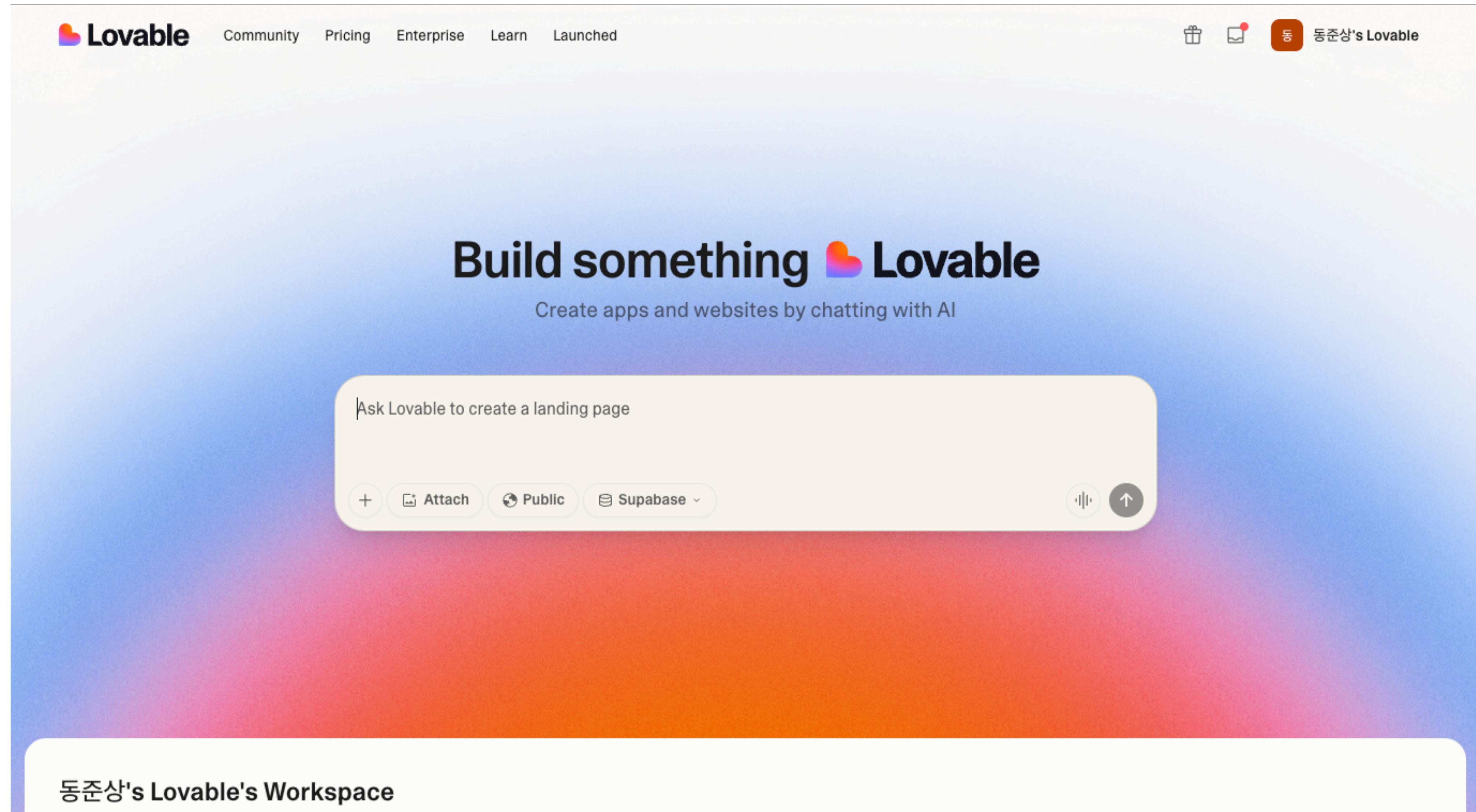
2. v0 (by Vercel)



3. Lovable

- 노코드 + AI 결합 웹앱 빌더
- 간단한 설명만으로 웹앱 자동 생성
- 구글 시트, Stripe 같은 외부 서비스 통합
- 비개발자도 SaaS 실험을 할 수 있는 최적의 도구

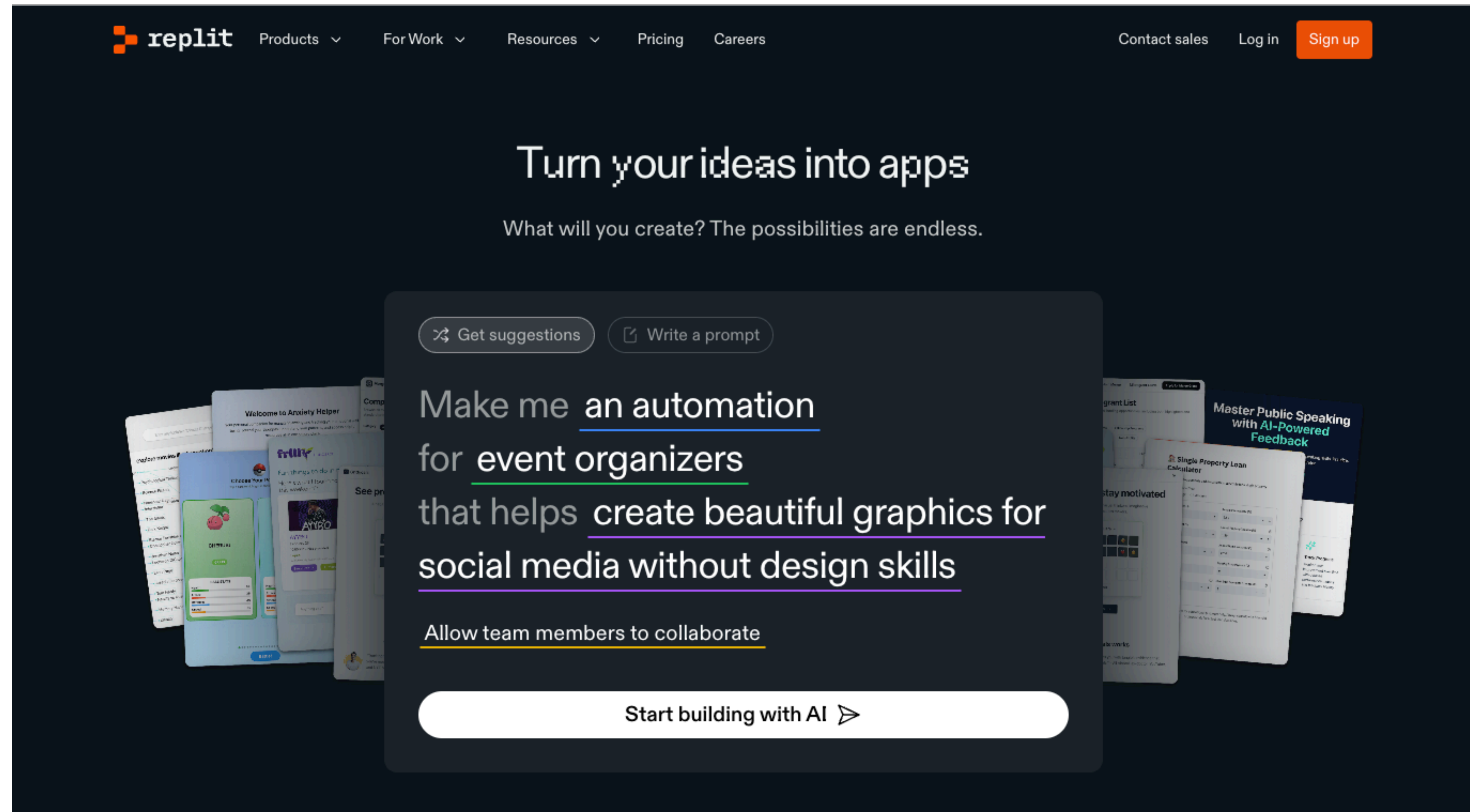
3. Lovable



4. Replit + Ghostwriter

- 온라인 IDE + AI 코드 어시스턴트
- 브라우저에서 협업 코딩, 멀티 언어 지원
- 배포까지 올인원 지원
- 학습자·팀 프로젝트에서 환경 세팅 없이 바로 실습 가능

4. Replit + Ghostwriter



The image shows the Replit website interface. At the top is a dark navigation bar with the Replit logo and links for Products, For Work, Resources, Pricing, and Careers. On the right are links for Contact sales, Log in, and a prominent orange Sign up button. The main section has a dark background with the headline 'Turn your ideas into apps' and the subtext 'What will you create? The possibilities are endless.' Below this is a central dark box containing a text input area with two buttons: 'Get suggestions' and 'Write a prompt'. The input text reads: 'Make me an automation for event organizers that helps create beautiful graphics for social media without design skills'. Below the input is a link 'Allow team members to collaborate' and a large white button that says 'Start building with AI' followed by a right-pointing arrow. To the left and right of the central box are stacks of colorful app mockups, including one titled 'Welcome to Anxiety Helper' and another 'Master Public Speaking with AI-Powered Feedback'.

replit Products ▾ For Work ▾ Resources ▾ Pricing Careers Contact sales Log in Sign up

Turn your ideas into apps

What will you create? The possibilities are endless.

Get suggestions Write a prompt

Make me an automation for event organizers that helps create beautiful graphics for social media without design skills

[Allow team members to collaborate](#)

Start building with AI ➤

5. Claude Code, Gemini Code

- AI 모델을 “코딩 동반자”로 쓰는 CLI 도구
- 터미널 기반 프롬프트 → 코드/스크립트/자동화 제안
- Cursor와 연계하면 강력한 듀오
- 바이트코딩 자동화 파이프라인에 적합 (예: 크롤러, 배치 작업, API 연결)

5. Claude Code

ANTHROPIC

Claude ▾API ▾Solutions ▾Research ▾Commitments ▾Learn ▾News

Try Claude

Your code's new collaborator

Unleash Claude's raw power directly in your terminal. Search million-line codebases instantly. Turn hours-long workflows into a single command. Your tools. Your workflow. Your codebase, evolving at thought speed.

Try Claude Code on Max

[See our pricing options](#)

A graphic of a terminal window with a dark background. Inside the terminal, the words "CLAUDE" and "CODE" are displayed in a large, orange, pixelated font. The letters are composed of small squares, giving it a retro, digital appearance. The terminal window has a standard macOS-style title bar with red, yellow, and green window control buttons in the top left corner.

5. Claude Code

ANTHROPIC

English ▾

Q Search...

⌘K

Research

Login

Support

Discord

Sign up >

⚙

Welcome

Developer Platform

Claude Code

Model Context Protocol (MCP)

API Reference

Resources

Release Notes

Build with Claude

Learn how to get started with the Anthropic API, the Console, and Claude Code.

Ask Claude about docs...

Developer Platform



Get started

Make your first API call in minutes.



Features overview

Explore the advanced features and capabilities now available in Claude.



API reference

Integrate and scale using our API and SDKs.







Ask AI AI

입문자 도구 조합 비법

- 핵심 IDE: Cursor
- UI/웹 생성: v0, Lovable
- 온라인 협업·학습: Replit
- CLI/에이전트형 개발: Claude Code, Gemini Code
- 구현 아이디어만 있다면, 해커톤 현장에서 5대 도구 조합으로 30분, 1시간, 3시간만에 프로토타입 개선하고 배포

VibeCoding Master

모범적인 바이브코딩 방식

(Best Practices for VibeCoding)



이미 검증된 몇 가지 바이트코딩 패턴

- “혼자·빠르게·반복 배포”로 수요 검증 후 자동화
- AI는 전문성 재배치: 문맥·평가·전환 역량 강화
- 맹목 신뢰 금지, 반복 검증으로 신뢰 형성
- 개인 기여자(Individual Contributor)로서 창의·재미 우선

선배 바ای브코더: 피터 레벨스의 패턴

- 자금 조달 없이 마이크로/미니 스타트업 반복 출시
- 속도·실행력 극대화, 관료주의 우회
- 니치 문제 → 간단한 스택으로 MVP
- 수요 확인 후 자동화/최적화 단계 진입
- Nomad List, Remote OK, Photo AI

아이디어 발굴1: 문제 포착

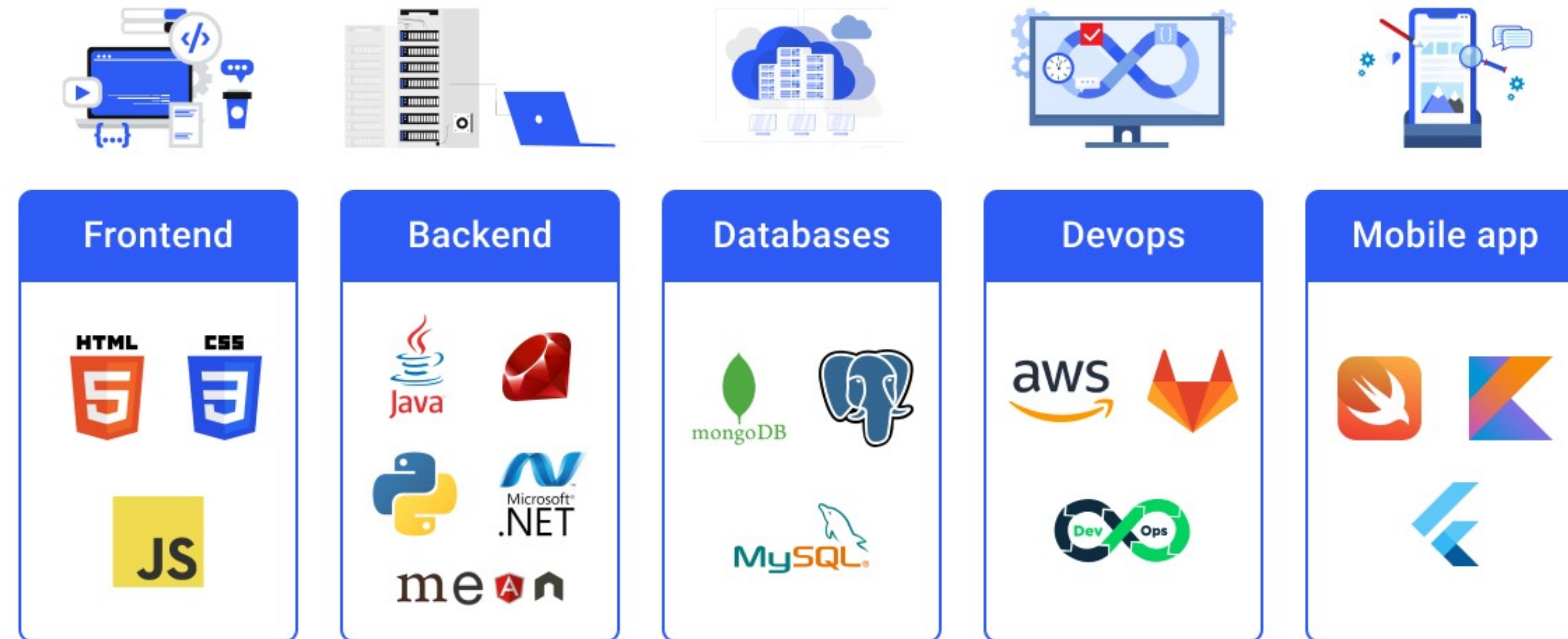
- 일상 불편을 목록화(Inbox·링크·정리 등)
- 자신에게 메시지(텔레그램/노트 앱)로 즉시 기록
- Trello/스프레드시트 등 가벼운 도구로 정리
- 잠재의식 숙성 → 타이밍 왔을 때 빠르게 착수

아이디어 발굴2: 기술/지리적 차이

- 기술 트렌드에서 가능성 탐색(Stable Diffusion, GPT)
- 다른 지역의 성공 모델을 로컬 문제에 적용
- 예: Photo AI—생성형 이미지의 실사용 가치 발굴
- 작은 실험으로 리스크 저비용 검증

기술 기반 선택: 웹앱, 모바일앱

Web Development Tech Stack



academysmart.com

iOS app development

Programming languages:



Objective-C



Swift

Toolkit:



Apple Xcode

SDK:



iOS SDK

Android app development

Programming languages:



Java



Kotlin

Toolkit:



Android Studio
& Android
Developer Tools

SDK:



Android SDK

Cross-platform app development

Programming languages:



React Native



Flutter

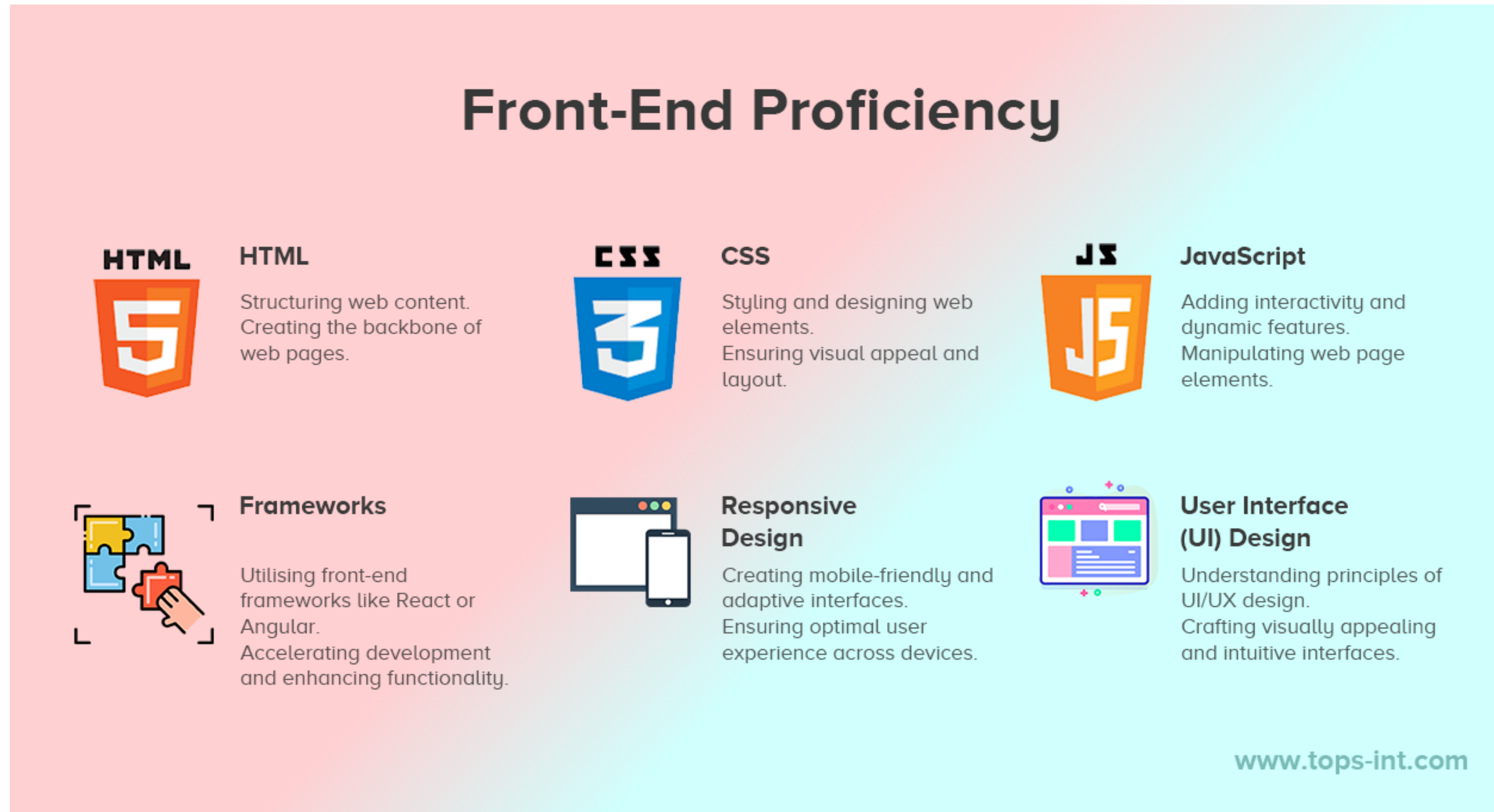


Unity

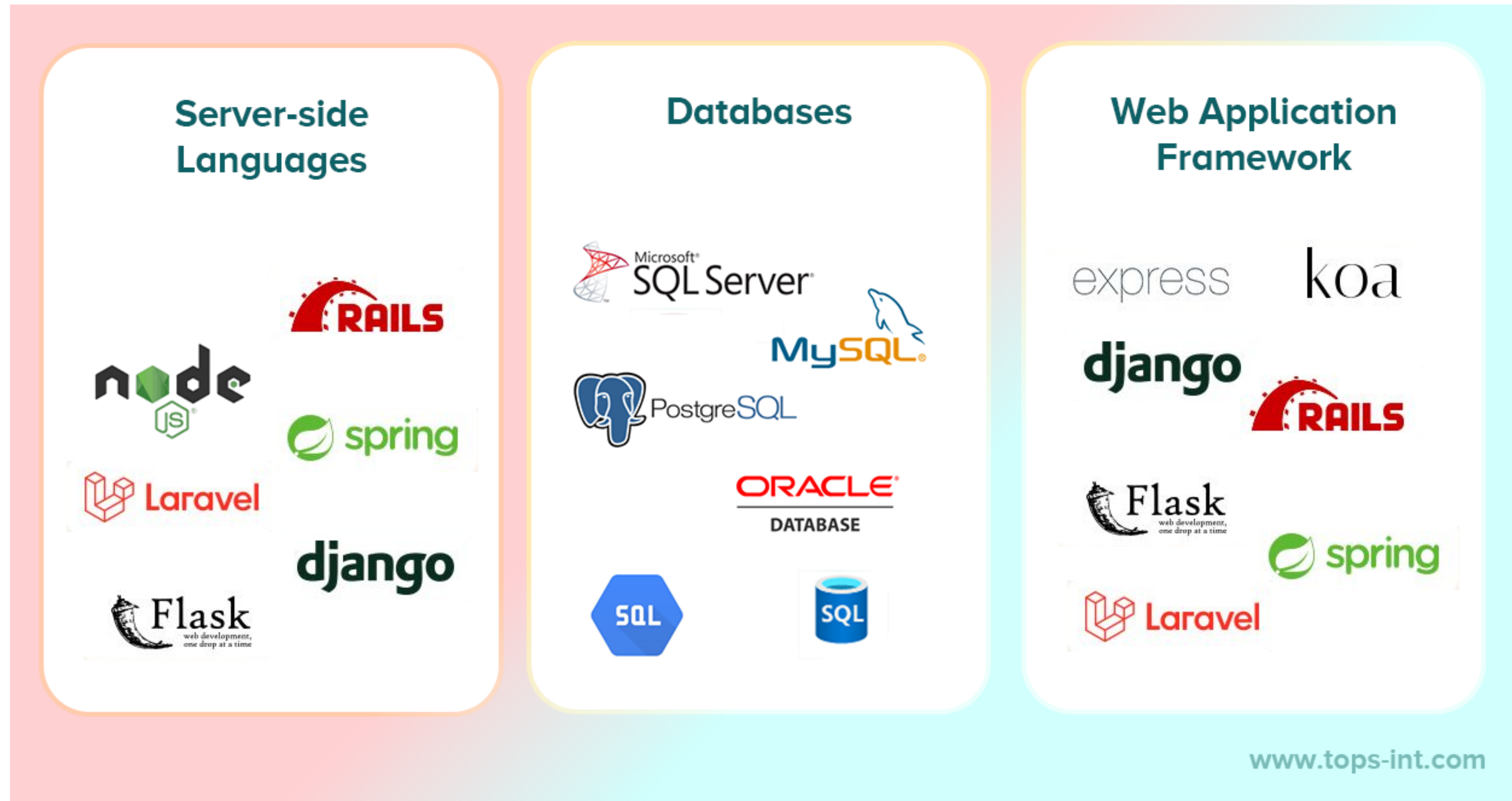


academysmart.com

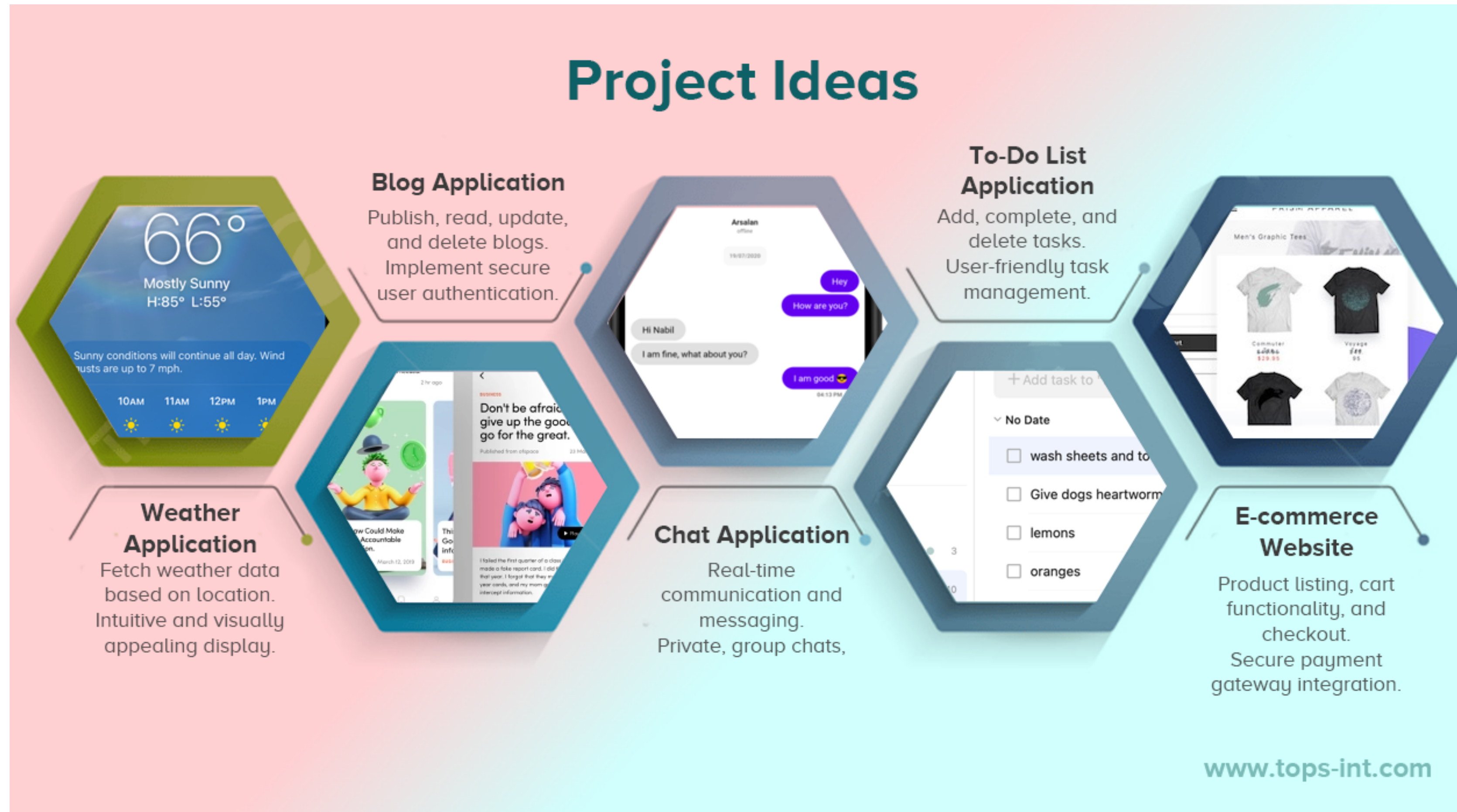
기술 기반 선택: 프론트엔드



기술 기반 선택: 백엔드 (서버&DB)



프로젝트 유형별 기술 기반



빌드 전략2: 내 주변 데이터부터 수집

- 초반엔 수동 처리로 수요 검증
- 유저 흐름 관찰 후 자동화 타이밍 판단
- 스프레드시트로 크라우드소싱(데이터/피드백)
- 예: Nomad List 초기 구글 시트 데이터 수집

신제품 출시 전략: 2주 내 출시

- 2주 이내 마이크로 제품 공개 목표
- 결제 의사(카드 입력)로 시장성 확인
- Building in Public: X/블로그로 투명 공유
- Shorts/TikTok/Reddit/HN로 유기 확산

성장 전략1: 피드백 루프 활용

- 부정적 피드백 포함, 모든 피드백에서 핵심 추출
- 포럼·디스코드로 커뮤니티 제안/토론
- 실사용 데이터로 우선순위 재조정
- 작은 개선의 빠른 릴리스 반복

성장 전략2: 데이터 실험

- A/B 테스트로 UI/파라미터 검증
- 예: Photo AI—생성 파라미터 무작위 테스트(10%)
- 선호 지표 학습 → 모델/UX 개선
- 유저가 제품을 스스로 개선하게 설계

수익화 원칙: 유료 우선

- 무료→유료 전환보다 처음부터 유료
- 인디 해커 기준 월 \$30+ 유의미 가격
- 고품질 고객군 확보·이탈 관리 용이
- 유료 플랜/업셀로 마진 극대화

수익 모델 포트폴리오

- 구독+일회성 결제 혼합
- 채용 보드·공지 유료화(Remote OK식)
- 템플릿/마켓 판매(프롬프트·리소스)
- 제휴/광고는 보조 채널로 운영

자동화1: 운영 자동화

- 성공 지표 확인되면 최대한 자동화
- 크론 잡으로 정기 배치(갱신/청소/레포트)
- Nomad List: 모임 조직/DM/트윗 자동화
- Webhook 기반 이벤트 처리 파이프라인

자동화2: 고객관리 자동화

- GPT, Gemini로 커뮤니티 필터링(스팸/욕설/밈 맥락)
- 리뷰/콘텐츠의 미묘한 뉘앙스 판단
- 사람이 하던 질적 판단도 보조 자동화
- 운영자 개입 시간 최소화

모니터링 & 헬스체크

- 전용 healthcheck 페이지로 상태 가시화
- DB 쿼리/속도/가입·결제 등 핵심 시그널
- Uptimebot 등으로 1분 주기 확인
- JS/PHP 오류를 텔레그램 즉시 알림

기술 스택 & 배포 문화

- 단순 스택 고수: “PHP는 그냥 작동”
- 새 프레임워크 과대홍보 경계
- Direct to Production: 스테이징 생략
- Lint/테스트+신속 롤백로 리스크 관리

프롬프트·컨텍스트 운용

- 요구사항 문서화(코드 가이드)로 환각 최소화
- 시스템 지침에 언어/프레임워크/제약 명시
- 한 번에 한 단계, 범위 축소 요청
- 새 세션/탭으로 문맥 초기화·리셋

하이브리드 디버깅 전략

- 브라우저 콘솔/네트워크/터미널 우선 확인
- 오류 메시지 그대로 AI에 투입해 패치 요청
- 모델 전환으로 응답 다양성 활용
- 복잡 버그: 가설→구체 수정 지시

문서화 & 거버넌스

- 코드·문서 버전 일치 점검
- 배포 전 PR 리뷰로 품질 최종 확인
- 약관/개인정보/환불 등 정책 문서 준비
- 최소 로깅·알림·에러 핸들링 표준화

개인 생산성 환경 설정 및 개선

- (레벨스의 경우) 1대 맥북·단일 화면 집중도 극대화
- 브라운 노이즈/테크노로 몰입
- 카페 등 제3의 공간에서 창의성 촉발
- 일반 조언보다 자기 맞춤 루틴 탐색(단식/운동 등)

결론 & 실행 체크리스트

- 작은 시작 → 2주 내 런치 → 데이터로 개선
- 유료 우선·마진 중심·간단 스택
- 수요 신호 확인 즉시 자동화로 스케일
- “Listen to nobody”—당신만의 바이브 구축

7 Best Practices for Vibe Coding

Bringing software discipline to AI-assisted development

1

Start with docs

Write requirements before code

2

Test first

Define behavior with tests

3

Use templates

Start with proven patterns

4

Precise prompts

Be specific in your instructions

5

Verify output

Always review generated code

6

Refactor regularly

Cleanup at each iteration

7

Measure constantly

Track metrics to improve

<https://www.kapihq.com/blog/vibe-coding-best-practices>

VibeCoding Master

바이브코딩 **입문** 프로젝트

(VibeCoding Entry Projects)



바이브코딩 입문 프로젝트 목록

1. 개인 랜딩 페이지 생성기
2. 뉴스 크롤러 & 카드뷰 요약
3. 유튜브 트렌드 대시보드
4. 간단한 To-Do & 메모 앱
5. 날씨/미세먼지 알리미
6. 이미지 프로필 (Photo AI)
7. 블로그 자동 발행기 (노션)
8. 이메일 뉴스레터 자동발송
9. 채용 게시판 (RemoteOK)
10. 헬스체크 모니터링 페이지

1. 개인 랜딩 페이지 생성기

- 업로드 자료: 자기소개 텍스트(이름, 관심사, SNS 링크)
- 기대 결과: index.html 파일, GitHub Pages/Cloudflare Pages에 배포된 개인 사이트

프롬프트 예시

HTML과 CSS로 1페이지 개인 랜딩페이지를 만들어줘.
섹션은 [프로필 사진 / 소개 / SNS 링크]로 구성하고,
모바일에서도 잘 보이도록 해줘.

2. 뉴스 크롤러 & 카드뷰 요약기

- 업로드 자료: RSS 피드 URL 또는 뉴스 기사 링크 모음
- 기대 결과: 뉴스 목록이 카드 형태로 표시되는 간단한 웹 페이지

프롬프트 예시

Python과 BeautifulSoup을 이용해
입력된 RSS URL에서 뉴스 제목과 요약을 가져와
카드 형태로 보여주는 웹페이지를 만들어줘.

3. 유튜브 트렌드 TOP10 대시보드

- 업로드 자료: 유튜브 API 키
- 기대 결과: API 속에 포함된 방대한 정보를 원하는 방식으로 가져오고 화면으로 제공하는 경험 증대

프롬프트 예시

YouTube API를 이용해
인기 영상 10개를 가져와서
제목, 썸네일, 조회수를 표로 표시하는 웹페이지를 만들어줘.

4. 간단한 To-Do & 메모 앱

- 업로드 자료: 없음
- 기대 결과: 간단한 구성 요소를 지닌 애플리케이션의 내부 구조를 살펴보며 이해도를 높일 수 있다.

프롬프트 예시

HTML, CSS, JavaScript로 할 일(To-Do) 리스트 앱을 만들어줘.
사용자가 할 일을 입력하고 완료 체크박스를 누르면
취소선이 표시되도록 해줘.

5. 날씨/미세먼지 알리미

- 업로드 자료: 공공데이터 API 키(예: 기상청, 미세먼지 API)
 - 기대 결과: 특정 도시의 실시간 날씨/미세먼지 정보 제공
- 페이지 구현

프롬프트 예시

OpenWeatherMap API를 이용해
사용자가 입력한 도시의 날씨와 미세먼지 지수를 보여주는
간단한 웹앱을 만들어줘.

6. AI 기반 이미지 프로필 생성기

- 업로드 자료: 개인 사진 1~2장
- 기대 결과: 사용자의 사진을 아바타/캐릭터 스타일로 변환된 이미지 출력

프롬프트 예시

Stable Diffusion API(Replicate)와 연동해
업로드한 사진을 아바타 이미지로 변환하는
간단한 웹 서비스 코드를 만들어줘.

7. 블로그 자동 발행기 (Notion → 웹)

- 업로드 자료: Notion API 키, 블로그용 데이터베이스
- 기대 결과: Notion 글이 자동으로 웹페이지에 반영되는 블로그 포스팅 자동화

프롬프트 예시

Notion API에서 글 데이터를 불러와서
정적 웹페이지(blog.html)로 변환하는
Python 스크립트를 작성해줘.

8. 이메일 뉴스레터 발송 자동화

- 업로드 자료: 이메일 수신자 리스트(CSV)
- 기대 결과: CSV 리스트에 저장된 사용자들에게 뉴스레터 자동 발송

프롬프트 예시

Gmail API를 사용해
뉴스 요약 텍스트를 자동으로 이메일 발송하는
Python 스크립트를 만들어줘.

9. 간단한 채용 게시판 (RemoteOK)

- 업로드 자료: 샘플 채용 공고 CSV
- 기대 결과: 구직자들이 검색/필터링 가능한 간단한 채용 사이트

프롬프트 예시

HTML과 PHP로
CSV에 있는 채용 공고를 보여주는 간단한 채용 게시판을 만들어줘.
검색 기능과 필터(지역, 직무)를 추가해줘.

10. URL 헬스체크 모니터링

- 업로드 자료: 샘플 API 엔드포인트(URL)
- 기대 결과: 특정 URL의 정상/오류 상태를 표시하는 대시보드

프롬프트 예시

주어진 API URL을 1분마다 요청해서
상태(200/500 등)를 체크하고 결과를 웹페이지에 표시하는
간단한 모니터링 앱을 만들어줘.

VibeCoding Master

바이브코딩 **응용** 프로젝트

(VibeCoding Advanced Projects)



바이트코딩 응용 프로젝트 목록

1. AI 요약 크롬 확장 프로그램
2. 쿠팡/아마존 리뷰 챗봇
3. 블로그 포스팅 봇 (크롤링)
4. 회의록& 태스크 생성 웹앱
5. AI 면접 연습 시뮬레이터
6. 프롬프트 마켓플레이스
7. 주식 리포트 이메일 발송기
8. 유튜브 숏폼 자동 자막 생성
9. 헬스체크 + Slack 대시보드
10. 결제 연동 (Stripe/Toss)

1. AI 요약 크롬 확장 프로그램

- 업로드 자료: 크롬 확장 기본 manifest.json 템플릿
- 기대 결과: 웹페이지 요약을 실시간으로 제공하는 확장 프로그램

1. manifest.json 기본 구조 생성
2. popup.html + popup.js 구현
3. GPT API 연동
4. 확장 로드 후 테스트
5. 버그 수정 및 배포

프롬프트 예시

크롬 확장 프로그램을 만들어줘.
버튼 클릭 시 현재 페이지 텍스트를 추출하고
GPT API를 호출해 3줄 요약을 보여주도록 해줘.

2. 쿠팡/아마존 리뷰 챗봇

- 업로드 자료: 리뷰 페이지 URL
 - 기대 결과: 리뷰 분석 후 사용자 질문에 답하는 챗봇
1. 리뷰 크롤러 코드 작성
 2. GPT API 연동
 3. 간단한 채팅 UI 구현
 4. 긍정/부정 라벨링 적용
 5. 테스트 및 배포

프롬프트 예시

JavaScript로
쿠팡 상품 페이지의 리뷰를 크롤링하고,
GPT API로 긍정/부정 요약 챗봇을 만들어줘.

3. 자동 블로그 포스팅 봇

- 업로드 자료: RSS 피드 URL
 - 기대 결과: 크롤링→요약→발행이 자동화된 블로그 봇
1. RSS 피드 파싱
 2. GPT 요약 적용
 3. Notion API 발행 기능 추가
 4. 스케줄러(cron) 설정
 5. 에러 핸들링

프롬프트 예시

RSS 피드에서
새 글을 가져와 요약 후, 마크다운으로 변환해
Notion API를 통해 자동 발행하는 파이썬 스크립트를 작성해줘.

4. 회의록 요약 & 태스크 생성 웹앱

- 업로드 자료: 음성 녹음 파일(mp3)
 - 기대 결과: 회의록 요약 + 할일 생성
1. 음성→텍스트 변환
 2. GPT 요약 적용
 3. 할 일(Task) 추출 기능 구현
 4. UI 구성
 5. 배포 및 공유

프롬프트 예시

AssemblyAI로 음성을 텍스트로 변환하고
GPT로 회의 요약 및 할 일(Task) 리스트를 생성하는
웹앱을 만들어줘.

5. AI 면접 연습 시뮬레이터

- 업로드 자료: 질문 리스트(예: CSV)
 - 기대 결과: 모의 면접 환경 제공
1. CSV 불러오기 기능 구현
 2. 랜덤 질문 출력
 3. 사용자 입력/음성 답변 수집
 4. GPT 평가 연동
 5. 결과 요약 및 저장

프롬프트 예시

면접 질문 CSV를 불러와 랜덤 질문을 출력하고,
사용자가 답변하면 GPT가 평가와 피드백을 주는
웹앱을 만들어줘.

6. 프롬프트 마켓플레이스 (SaaS)

- 업로드 자료: 샘플 프롬프트 10개
 - 기대 결과: 간단한 PromptBase 스타일의 마켓
1. 업로드/검색 기능 구현
 2. 데이터 저장(DB/JSON)
 3. UI 구성
 4. 공유 버튼 추가
 5. 배포

프롬프트 예시

사용자가 프롬프트를 올리고
검색/다운로드할 수 있는
간단한 마켓 웹서비스를 만들어줘.

7. 자동 리포트 이메일 발송기

- 업로드 자료: 주식/환율 API 키
 - 기대 결과: 간단한 PromptBase 스타일의 마켓
1. API 데이터 수집
 2. GPT 요약 적용
 3. 이메일 발송 기능 구현
 4. 스케줄링(cron) 설정
 5. 테스트 및 배포

프롬프트 예시

매일 아침 9시에 주식/환율 데이터를 가져와
GPT 요약 보고서를 작성해 이메일로 발송하는
스크립트를 만들어줘.

8. 슷폼 자동 자막 생성기

- 업로드 자료: 유튜브 영상 파일
- 기대 결과: 자동 자막 생성 & 동영상 적용

1. 영상 음성 추출
2. Whisper API 적용
3. srt 자막 파일 생성
4. 동영상에 자막 입히기
5. 결과 확인

프롬프트 예시

OpenAI Whisper로
영상 음성을 텍스트로 변환하고,
자막(srt) 파일을 생성해주는 코드 작성해줘.

9. 헬스체크 + Slack 알림 대시보드

- 업로드 자료: API 엔드포인트 목록
 - 기대 결과: 실시간 모니터링 대시보드 + Slack 알림
1. API 상태 체크 코드 작성
 2. 결과 대시보드 구현
 3. Slack API 연동
 4. 오류 시 알림 전송
 5. 배포

프롬프트 예시

주어진 API 리스트를 1분마다 체크해 상태를 대시보드에 표시하고, 오류 발생 시 Slack으로 알림을 보내는 웹앱을 만들어줘.

10. 유료 결제 연동 웹서비스

- 업로드 자료: Stripe/Toss API 키
 - 기대 결과: 결제 가능한 실전형 웹 서비스
1. 결제 버튼/폼 구현
 2. Stripe/Toss API 연동
 3. 결제 성공/실패 처리
 4. 구독 플랜 구성
 5. 배포 및 테스트

프롬프트 예시

Stripe API를 이용해
간단한 구독 결제 시스템을 구현하는
Node.js/Express 웹서비스 코드를 작성해줘.

VibeCoding Master

바이브코딩 점프 스타트

(Toolkits for VibeCoding Jump Start)



바이브코딩 스타터

- 바이브코딩 스타터 템플릿으로 활용
- 깃허브에는 수만 건 이상의 다운로드 기록을 가진 우수한 미니 프로젝트 리포지토리 존재
- 각 리포는 바이브코딩 시에 복제하여
- AI 중심 실습에 빠르게 통합

바이브코딩 스타터: 깃허브 리포 10선

- CMSaaSStarter: 프롬프트 마켓, 프리미엄 대시보드 확장
- chrome-extensions-samples: 크롬 생태계와 AI 통합
- Web Starter Kit: 반응형 랜딩 페이지 + 프롬프트 요약
- VisBug: AI 기반 CSS 제안 도구 개발
- Stylus: 사용자 스타일 추천 시스템 구현

바이브코딩 스타터: 깃허브 리포 10선

- Pagoda: Go 기반 자동 API 서버 확장 실험
- Yeoman: 맞춤 템플릿 생성 UI 구현
- tldr-pages: 명령어 요약 써머리 AI 챗봇 제작
- Telegram Mini Apps: Telegram 연동 AI 봇 구축
- Static Site Generators: 블로그 자동화 + GPT 요약 연동

바이브코딩 스타터

- 바이브코딩 스타터 템플릿으로 활용
- 깃허브에는 수만 건 이상의 다운로드 기록을 가진 우수한 미니 프로젝트 리포지토리 존재
- 각 리포는 바이브코딩 시에 복제하여
- AI 중심 실습에 빠르게 통합

VibeCoding Master

바이브코딩 **실전** 프로젝트

(VibeCoding Projects: CreatorMagic)



1. QuizForge

- 텍스트·키워드·이미지로 퀴즈 생성
 - 준비: 샘플 텍스트 파일, 구글 계정
 - 사용자 업로드 자료로 퀴즈 생성 + 화면 출력
1. HTML 업로드 버튼 구현
 2. 텍스트 읽기 후 GPT 호출
 3. 문제/보기/정답 출력
 4. JSON 구조(질문-보기-정답)
 5. 스타일링(Tailwind or CSS)

프롬프트 예시

사용자가 업로드한 텍스트 파일을 읽어 GPT API를 이용해 5개의 객관식 퀴즈를 생성하는 웹앱 코드를 작성해줘. 출제된 문제와 정답을 화면에 카드 형태로 표시해줘.

2. Doodely

- Cursor IDE와 AI 도구로 캐주얼 게임을 따라 만드는 미니 프로젝트
 - 제한 시간 내 점수를 올리는 간단한 브라우저 게임
1. HTML 버튼 + 점수 표시 영역
 2. JS 이벤트 리스너 추가
 3. 타이머(30초) 구현
 4. 종료 시 최종 점수 출력
 5. CSS로 UI 개선

프롬프트 예시

HTML/JavaScript로 간단한 클릭 게임을 만들어줘.
사용자가 버튼을 클릭할 때마다 점수가 올라가고,
30초 후 최종 점수를 표시하도록 해줘.

3. BabyName

- 사용자가 이름을 검색하면 인기 순위와 변동 추이를 시각화
 - Baby Names API, HTML/CSS 템플릿 파일
1. 입력창 + 검색 버튼
 2. API 호출 구현
 3. 순위 데이터 JSON 파싱
 4. 결과 표/차트로 출력
 5. 에러 처리 (없는 이름 입력)

프롬프트 예시

US Baby Names API를 사용해
특정 이름의 순위를 보여주는 검색형 웹앱을 작성해줘.
사용자가 이름을 입력하면 인기 순위와 연도별 변화를 표로 보여줘.

4. Reddit Trends Dashboard

- 레딧의 인기 게시물을 모아 보여주는 대시보드. 오늘 가장 많이 언급된 Reddit 게시물 대시보드 완성
 - Reddit API 키, Chart.js, D3.js
1. API 호출(Hot/Trending)
 2. JSON 데이터 파싱
 3. 게시물 카드뷰 표시
 4. 차트 라이브러리 연결
 5. 스타일링 및 반응형 디자인

프롬프트 예시

Reddit API를 이용해
오늘의 인기 게시물 10개를 불러와
제목·추천수·댓글수를 표와 차트로 표시하는 대시보드를 만들어줘.

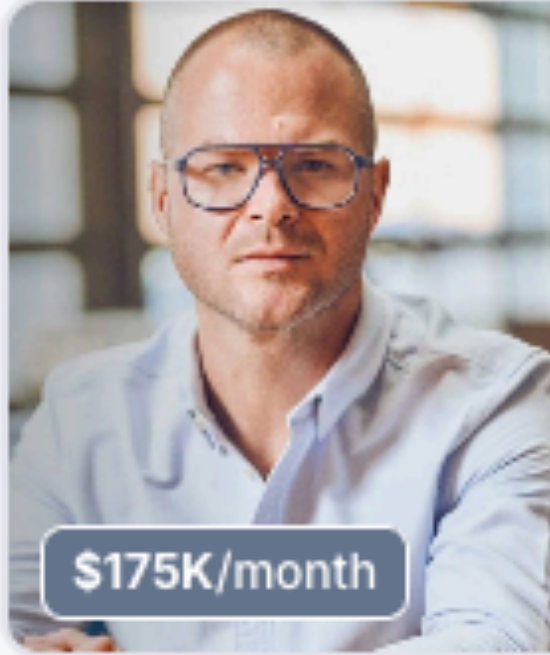
5. Focus - Pomodoro Timer

- 브라우저 기반 25분 집중 + 5분 휴식 루프를 제공하는 타이머 앱
 - HTML/CSS 기본 템플릿, JavaScript 기초 문법
1. 타이머 UI (분:초 표시)
 2. Start/Stop 버튼 구현
 3. 25분/5분 주기 루프 구현
 4. 사이클 카운트 표시
 5. localStorage 저장 기능 추가

프롬프트 예시

Pomodoro 기법 타이머를 HTML/CSS/JavaScript로 만들어줘.
25분 집중 / 5분 휴식 루프가 반복되며,
완료된 사이클 횟수를 화면에 표시하도록 해줘.

참고자료: 스타터 스토리 (니치 마켓)



How I Make \$2M/Year As A Solopreneur

Justin breaks down:

- ✓ Setting up a one person business
- ✓ Creating a scalable content machine

\$175K/month

case study



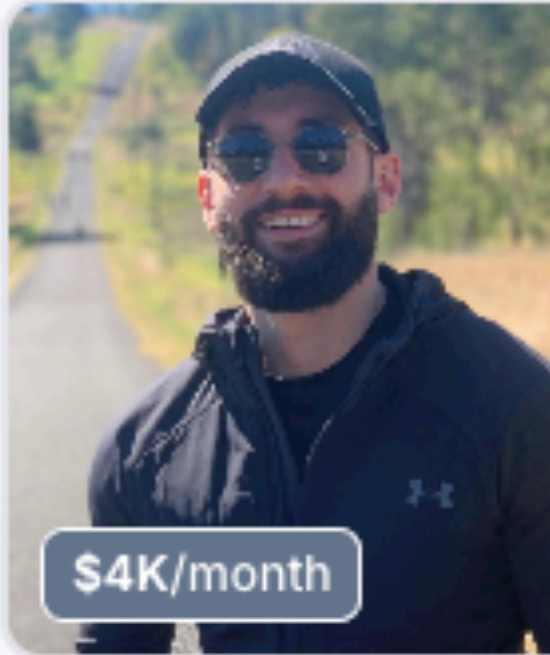
How I Started A \$144M Brand At 23 Years Old

Dylan breaks down:

- ✓ Validating a product with FB ads
- ✓ Finding a supplier that can scale

\$12M/month

case study



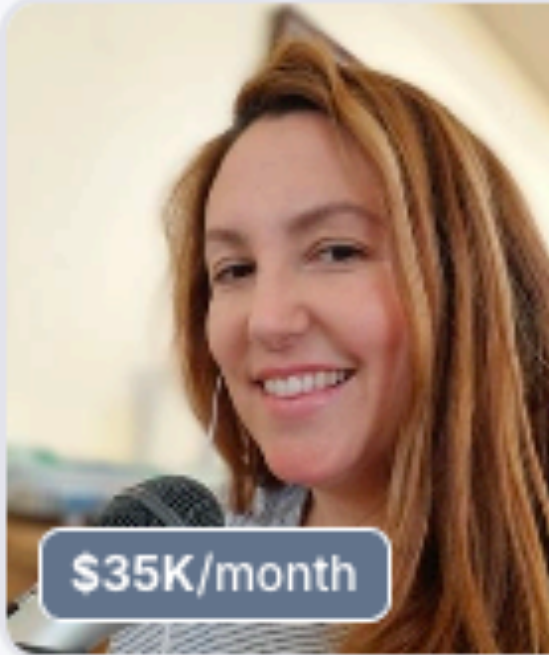
How My Product Hunt Launch Generated \$5.8K

Luc breaks down:

- ✓ Launching on Product Hunt
- ✓ Creating a profitable newsletter

\$4K/month

case study



How I Built A \$420K/Year Productized Service

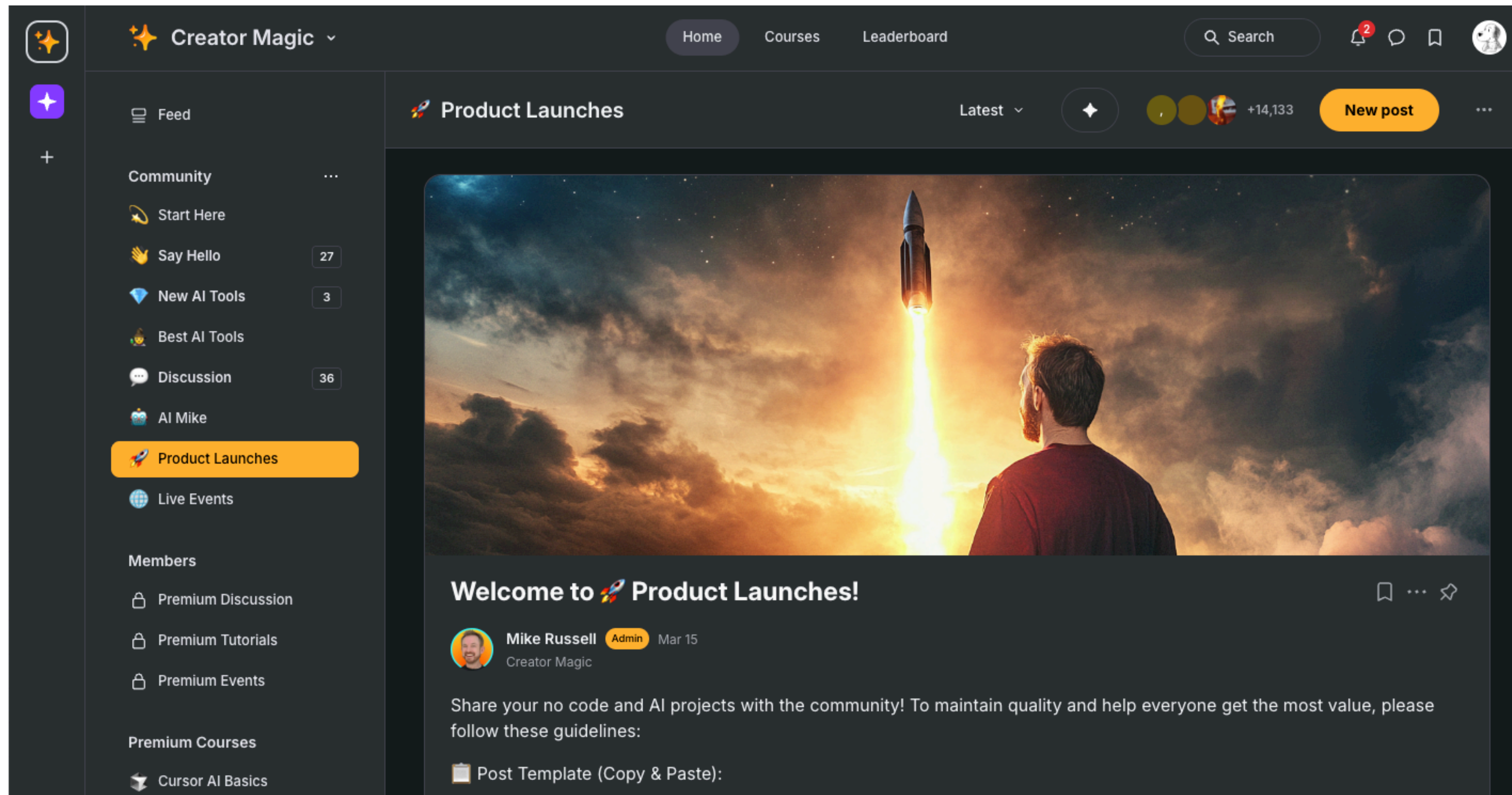
Jaclyn breaks down:

- ✓ Setting up a productized service
- ✓ Landing first client with cold email

\$35K/month

case study

참고자료: 크리에이터 매직 (확장)



<https://members.creatormagic.ai/c/product-launches/>

VibeCoding Master

바이트코딩의 한계영역

(Gray Area of VibeCoding)



바이트코딩의 4대 한계 유형

- 소프트웨어 엔지니어링 유형별 한계
- 프로그래밍 언어 관점에서의 한계
- 프레임워크/라이브러리 관점에서의 한계
- 업무맥락적/조직문화적 제약 조건

SW: 미션 크리티컬 시스템



- 항공·철도 관제, 군사 시스템, 금융 코어 बैं킹, 원자력 제어
- 실시간성·안전성·법규 준수가 최우선, AI 코드 생성의 “불확실성”을 허용하기 어려움
- 코드 1줄 오류가 막대한 사고로 이어질 수 있어 “실험적·물입형 개발”보다 형식 검증(Formal Verification), 인증 프로세스가 요구됨

SW: 고성능/임베디드 시스템



- 반도체 펌웨어, 의료기기 내장 SW, 차량 ECU 등
- 리소스 제약(메모리, 전력) + 실시간 스케줄링 요구 → AI 자동생성 코드가 최적화된 C/어셈블리 수준을 보장하기
힘듦

SW: 대규모 분산/레거시 통합



- 20년 이상 된 레거시 시스템(COBOL, PL/I 등)과 연계
- AI가 코드를 제안해도 레거시 환경에 배포/테스트하기 어려워 **바이브코딩 루프(프롬프트→코드→실행)**가 깨짐

SW: 보안·규제 필수 영역



- 전자정부, 국방, 공공데이터 보안망
- 오픈AI API나 외부 IDE에 연결 불가 → 로컬 LLM + 온프레미스 IDE 도입 없이는 활용 곤란

프로그래밍 언어 관점

- 어셈블리, VHDL(HW 설계), Fortran, COBOL: 언어 자체가 저수준·도메인 특화, 실습·실행 환경이 일반 PC와 다름 
- 특수 목적 DSL(Domain-Specific Language): SAP ABAP, Oracle Forms 등 → 학습 데이터 희소성, 폐쇄적 생태계 
- Python, JavaScript, TypeScript, Go는 바이트코딩에 매우 적합 (풍부한 예제, 즉시 실행 환경, 빠른 배포 가능) 

프레임워크/라이브러리 관점



- 임베디드/실시간 OS용 프레임워크: FreeRTOS, QNX → 타이밍 제약
- EDA/시뮬레이션 라이브러리: Synopsys, Cadence → 상용, 폐쇄적, 복잡한 라이선스
- 고성능 HPC 프레임워크: MPI, OpenMP → 분산 환경 세팅 필요, 클라우드 실습 불가 시 어려움

프레임워크/라이브러리 관점



- 웹/앱 프레임워크: React, Next.js, SvelteKit, Flask, Django
- 자동화/데이터 파이프라인: LangChain, Airflow, Supabase, Firebase
- 크롬 확장·노코드 API 연동: Vercel, Netlify, Zapier

맥락적/문화적 제약



- 대기업/공공기관 망분리: API 호출 불가 → 바이트코딩 도입 어려움
- 개발 문화: “실험적 접근”보다 문서화·검증·심사 프로세스 우선
- 책임 구조: 오류 발생 시 책임 소재가 모호 → AI 코드 제안 신뢰도 낮음

결론: 직접적 소비자 니즈 충족



- 빠른 실험과 창의적 제품
- 다양한 의견 수렴 빌드
- 개인 창작자, 스타트업, 학습자 중심 폭발적 성장



- 안정·고신뢰·고규제 환경
- 대기업, 산업 제어, 미션 크리티컬, 보안 규제, 레거시 통합 영역

goorm



AI Fit-T 프로젝트 : 바이트코딩 (25.09.05~10.02)

Cursor, Code, v0 바이트코딩 20제 마스터